

CTA200H Assignment 3

ZI JIAN BRENDAN PENG

2026-05-12, 13:00

§1 Question 1

A function `convergence(c, max_iter = 10, radius = 2)` was defined to check if a complex number defined by the coordinates at that point caused the recursive formula defined by $z_0 = 0, z_{n+1} = z_n^2 + c$ to converge or diverge. `c` is a `numpy.meshgrid` object that contained each complex number on the xy plane defined by $c = x + iy$. A radius of 2 was found to suffice to check if a given value z diverged or converged, as it was found after some basic trial and error that all values that converged had an absolute value of approximately less than 2. Using larger values for radius resulted in the same figures being produced.

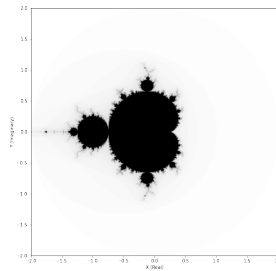


Figure 1: Divergence (white) and convergence (black) of different points $c = x + iy$, when used in the recursive formula $z_0 = 0, z_{n+1} = z_n^2 + c$

The `convergence()` function returned a `numpy.meshgrid` object that contained the iteration `n` at which point $\text{abs}(z) > 2$. Figure 1 Was produced using the `pyplot.imshow()` function, plotting the output of `convergence(c, 100)`, and using the additional argument `cmap = "binary"` to produce a two colour image, as all points on the plot that do not diverge would be assigned a significantly higher value by the `convergence()` function and thus coloured black by `pyplot`.

As the `convergence()` function also returns the value of iteration where a given point z diverges, we can use `pyplot.contourf()` to plot a colourscale indicating at which iterations a given point diverges, producing figure 2. Points coloured yellow were all points that were not found to diverge within the `max_iter` iterations done by the program.

It can be seen here that most points that diverge, will diverge fairly early on in very few iterations.

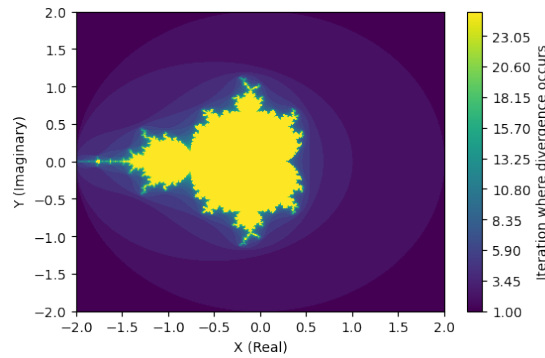


Figure 2: The value n for which the recursive formula $z_0 = 0, z_{n+1} = z_n^2 + c$ diverges

§2 Question 2

§2.1 Part 1

A function `lorenz(t, W, sigma, r, b)` was defined to find $\dot{W} = [\dot{X}, \dot{Y}, \dot{Z}]$, for a given $W = [X, Y, Z]$. W is a tuple with entries $W=(X,Y,Z)$, and `sigma`, `r`, and `b` are parameters defined in the Lorenz equations. `lorenz()` returns a tuple containing $\dot{X}, \dot{Y}, \dot{Z}$ as defined by the Lorenz equations. `t` is also there but it just serves as a placeholder so that the function can be used by `scipy.integrate.solve_ivp`.

§2.2 Part 2

The ODE is solved using the aforementioned `scipy.integrate.solve_ivp` function, and the initial values $W = (0.0, 1.0, 0.0)$ and `sigma = 10.0`, `r = 28`, `b = 8.0/3.0` are used as the initial values. The additional argument `t_evals` is used with `t = numpy.linspace(0,60,6000)` as the inputted parameter, this is to allow for easier graphing later on when specific points or sections of the solution are graphed.

The solution to the ODE was stored to a variable `sol`.

§2.3 Part 3

Figure 3 is a recreation of figure 1 from Lorenz (1963) [1]. A `pyplot.subplots()` was used to plot the three sections of the solution to Y . Sections of the solution was obtained for each subplot by slicing the solution array (i.e. `sol.t[0:2000]`, `sol.y[1][0:2000]` to obtain the first third of the solution with respect to time).

The oscillations here can be seen to become more and more drastic as time increases.

§2.4 Part 4

Figure 4 is a refreation of figure 2 from Lorenz (1963) [1], which is a projection of the solution from 14 seconds to 19 seconds onto the YZ and YX planes. Note that because in the original paper the plane is graphed as YX instead of XY , that is what is used in this recreation as well.

Once again `pyplot.subplots()` was used and the period 14 to 19 seconds was obtained by slicing the list from entries `[1400:1900]`. Labels were then added by drawing a scatterplot `pyplot.scatter()`, picking the points corresponding the each whole integer second from 14 to 19. A for-loop was then used to enumerate through each of these points to add a label like in the original paper using `pyplot.annotate()`.

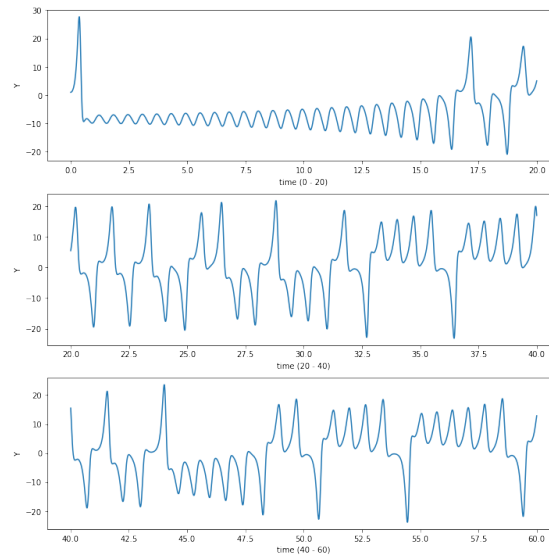


Figure 3: Recreation of figure 1 from Lorenz (1963) [1], showing the solution for Y with respect to time t

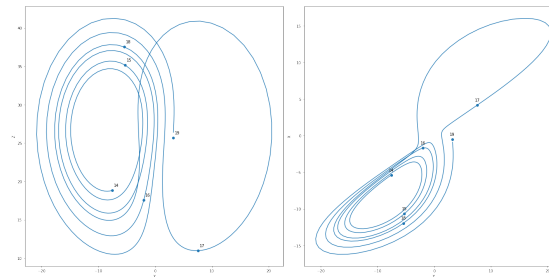


Figure 4: Recreation of figure 2 from Lorenz (1963) [1], projection of the solution of W onto the YZ plane and the YX plane

§2.5 Part 5

A second solution to the ODE was calculated, this time using initial values $W' = [0.0, 1.0 + 1 \times 10^{-8}, 0.0]$, obtained by adding the numpy array `numpy.array([0.0, 1.0e-8, 0.0])` to the original initial values W , taking advantage of the fact that numpy arrays do arithmetic by each individual entry. This solution was stored to a variable `solnew`. A new array was then created by subtracting the difference between `sol` and `solnew`, and then taking the absolute value at each time t . Note that the absolute value had to be manually calculated due to the way the array was configured. The absolute value of the difference was then plotted with respect to time, producing figure 5.

It can be seen that the two solutions appear to mostly agree with one another for the first 30 seconds, with the difference being close to 0. However after 30 seconds we see that small differences appear until suddenly the graph becomes extremely erratic with huge differences between the two solutions.

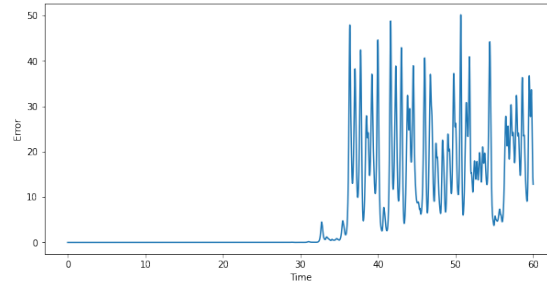


Figure 5: Graph of difference with respect to time for two differing initial values of $W = [0.0, 1.0, 0.0]$ and $W' = [0.0, 1.0 + 1 \times 10^{-8}, 0.0]$

References

- [1] Edward N. Lorenz. “Deterministic Nonperiodic Flow”. In: *Journal of Atmospheric Sciences* 20.2 (1963), pp. 130–141. DOI: [10.1175/1520-0469\(1963\)020<0130:DNF>2.0.CO;2](https://doi.org/10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2). URL: https://journals.ametsoc.org/view/journals/atsc/20/2/1520-0469_1963_020_0130_dnf_2_0_co_2.xml.